Technical Code Documentation: SyncShare Engine

Implementation Manual

I. Overview

This document serves as the technical reference for the SyncShare application. The architecture consists of the Data Engine (`NoteEngine.hpp`) and the Application Controller (`main.cpp`).

II. Data Engine (`NoteEngine.hpp`)

- **Class** `BaseNote`: Defines the fundamental data structure.
- **Operator Overloading**: Implements the `<<` operator as a friend function to allow direct stream output of note objects (`std::cout << myNote`), ensuring consistent formatting.
- **Class** `SecureNote`: Inherits from `BaseNote`, adding encryption fields.
- **Function** `logNoteAction`: A template function demonstrating generic programming to log actions for any note type.

III. Controller Logic (`main.cpp`)

- **Function** `sync_callback`: A recurring timer function (1.0s) that provides bidirectional sync by refreshing the GUI browser from the persistent `notes.txt` file.
- **Function** `add_note_cb`: Callback for the UI button to append data.
- **Command-Line Controller**: Parses `argc/argv` for non-GUI data maintenance (stats, import, export, delete).
- **Smart Pointers**: Utilizes `std::unique_ptr` to manage the lifetime of GUI components, ensuring safe memory management and prevention of leaks.

IV. Summary of Engineering Standards

The project follows modern C++ development practices:

- **RAII & Memory Management**: Use of smart pointers for automatic resource cleanup.

- **OOP Principles**: Implementation of inheritance and polymorphism.
- **Operator Overloading**: Used for clean, idiomatic output stream handling.
- **Error Handling**: Robust file stream verification using `is_open()` and stream validation.